

Generating RS256 JWK Keys in Ansible

<https://gemini.google.com/app/897c7632ec5a63f8>

User prompt: any ansible module that could generate rs256 jwt keys like this project <https://github.com/bspk/json-web-key-generator>

Response: There is **no native built-in Ansible module or official community.crypto module** that directly outputs RFC 7517 JSON Web Keys (JWK) in the JSON format produced by [bspk/json-web-key-generator](https://github.com/bspk/json-web-key-generator). Ansible's crypto modules (like `community.crypto.openssl_privatekey`) only generate PEM/DER formats.

However, you can accomplish this cleanly in Ansible using one of the approaches below.

Option 1: Execute Python with jwcrypto (Recommended)

The Python library `jwcrypto` implements RFC 7517 and outputs the exact same structure (`key`, `alg`, `use`, `kid`, `n`, `e`, `d`, `p`, `q`, etc.).

1. Ensure jwcrypto is installed

```
- name: Install jwcrypto on target/controller
  ansible.builtin.pip:
    name: jwcrypto
    state: present
```

2. Generate the key pair

```
- name: Generate RS256 JWK keypair
  ansible.builtin.command: >-
    python3 -c "
    import json, uuid
    from jwcrypto import jwk
    key = jwk.JWK.generate(kty='RSA', size=2048, alg='RS256', use='sig', kid=str(uuid.uuid4()))
    output = {
        'private_jwk': json.loads(key.export(private_key=True)),
        'public_jwk': json.loads(key.export(private_key=False))
    }
    print(json.dumps(output))
    "
  register: jwk_result
  changed_when: false

- name: Parse JWK JSON output
  ansible.builtin.set_fact:
    private_jwk: "{{ (jwk_result.stdout | from_json).private_jwk }}"
    public_jwk: "{{ (jwk_result.stdout | from_json).public_jwk }}"

- name: Save public JWKS file
  ansible.builtin.copy:
    dest: /path/to/jwks.json
    content: "{{ {'keys': [public_jwk]} | to_nice_json }}"
    mode: '0644'

- name: Save private JWK file
  ansible.builtin.copy:
    dest: /path/to/private_jwk.json
    content: "{{ {private_jwk | to_nice_json }}"
    mode: '0600'
```

Option 2: Custom Ansible Filter Plugin

If you want a native Jinja2 filter syntax like `{{ private_key_pem | to_jwk }}`, create a filter plugin at `filter_plugins/jwk_filter.py`:

```
import json
import uuid
from jwcrypto import jwk

def rsa_to_jwk(pem_data, kid=None, use="sig", alg="RS256"):
    key = jwk.JWK.from_pem(pem_data.encode("utf-8"))
    key_id = kid if kid else str(uuid.uuid4())

    priv_dict = json.loads(key.export(private_key=True))
    pub_dict = json.loads(key.export(private_key=False))

    for item in (priv_dict, pub_dict):
        item["kid"] = key_id
        item["use"] = use
        item["alg"] = alg

    return {
```

```

        "private": priv_dict,
        "public": pub_dict
    }

class FilterModule(object):
    def filters(self):
        return {
            "rsa_to_jwk": rsa_to_jwk
        }

```

Then in your playbook, pair it with `community.crypto.openssl_privatekey`:

```

- name: Generate standard RSA private key
  community.crypto.openssl_privatekey:
    path: /tmp/rsa_private.pem
    size: 2048
    type: RSA
    register: rsa_key

- name: Read private key content
  ansible.builtin.slurp:
    src: /tmp/rsa_private.pem
    register: raw_pem

- name: Convert to JWK format
  ansible.builtin.set_fact:
    jwk_keys: "{{ (raw_pem.content | b64decode) | rsa_to_jwk }}"

```

Option 3: Using Smallstep CLI (step-cli)

If you have `step` installed on the target host, it has a dedicated JWK generator command:

```

- name: Generate RS256 JWK pair using step-cli
  ansible.builtin.command: >-
    step crypto jwk create /tmp/pub.jwk /tmp/priv.jwk
    --kty RSA
    --size 2048
    --alg RS256
    --use sig
    --no-password
    --insecure
  args:
    creates: /tmp/priv.jwk

```

Sources

1. [How to generate a JSON Web Key \(JWK\) - Connect2id](#)

User prompt: first time seeing step-cli, explain to me

Response: **step** (commonly referred to as **Smallstep CLI** or `step-cli`) is an open-source, Swiss-Army-knife command-line utility for modern public key infrastructure (PKI), cryptography, and identity management. It is developed by [Smallstep](#).

Think of it as a modern, developer-friendly replacement for awkward `openssl` commands, specifically built around contemporary standards like **JSON Web Tokens (JWT)**, **JSON Web Keys (JWK)**, **OAuth/OIDC**, **ACME**, and **mutual TLS (mTLS)**.

What Makes It Different from openssl?

- **Native Modern Web Standards:** OpenSSL was created long before JWTs, JWKs, and OAuth existed. To inspect a JWT or export a JWK with OpenSSL, you usually need custom bash pipelines or Python scripts. `step` treats JWKs, JWTs, and JOSE structures as first-class citizens.
- **Sensible Defaults:** Command syntax follows modern CLI patterns (clear subcommands, human-friendly flags, JSON output flags) rather than cryptic legacy OpenSSL flags like `-in`, `-out`, `-nodes`, `-subj`.
- **Zero Dependencies:** Written in Go and distributed as a single static binary.

Core Areas of Functionality

1. JWK & Key Management (step crypto jwk)

It can generate, inspect, and convert cryptographic keys directly into RFC 7517 JWK/JWKS format:

```

# Generate a keypair directly into JWK files
step crypto jwk create pub.json priv.json --kty RSA --size 2048 --alg RS256 --no-password --insecure

# Convert an existing OpenSSL PEM file into a JWK
step crypto jwk create pub.json priv.json --from-pem /path/to/key.pem --no-password --insecure

```

```
# Generate a Key ID (kid) fingerprint from a JWK
step crypto jwk thumbprint pub.json
```

2. JWT Generation & Verification (step crypto jwt)

You can sign, verify, and inspect JWT tokens directly from the shell without writing application code:

```
# Sign a token with a JWK
step crypto jwt sign --key priv.json --iss "my-issuer" --sub "user-123" --aud "my-api"

# Inspect and decode payload without verifying (like jwt.io in your terminal)
step crypto jwt inspect <token>

# Verify a token against a public key or JWKS
step crypto jwt verify <token> --key pub.json --aud "my-api" --iss "my-issuer"
```

3. X.509 Certificates & Internal PKI (step certificate)

Simplifies generating self-signed certificates, certificate signing requests (CSRs), and full internal hierarchies:

```
# Create a root CA
step certificate create "Internal Root CA" root-ca.crt root-ca.key --profile root-ca

# Create a leaf certificate signed by that root
step certificate create internal.service.local service.crt service.key \
  --ca root-ca.crt --ca-key root-ca.key --profile leaf
```

4. Step CA Integration

Smallstep also provides `step-ca`, a lightweight, production-ready private certificate authority server. The `step` CLI serves as the administrative client to configure and manage automated certificate enrollment via ACME, OAuth/OIDC provisioners, and TPM-backed keys.

Common Use Cases in Infrastructure

- **Automation & CI/CD:** Quick creation of dummy or production JWKS files and test JWTs for API mocking, integration tests, and service-to-service auth pipelines.
- **Debugging Auth Flows:** Verifying signatures, checking token expiries, and validating JWKS endpoints directly on servers or bastion hosts.
- **Internal mTLS:** Bootstrapping microservice-to-microservice or homelab TLS encryption without dealing with manual OpenSSL config files.

User prompt: how to use step command to generate jwks key pair like this

Response: To produce that exact structure, note two specific details about your sample files:

1. **JWKS wrapper** (`{"keys": [...]}`): Standard `step crypto jwk create` outputs a standalone JWK object (`{ ... }`), not a keyset array. Piping into `jq` wraps it into `{"keys": [...]}`.
2. **RFC 7638 SHA-256 Base64 kid:** The `kid` ending in `\u003d` (an escaped `=` sign) is a standard Base64-encoded SHA-256 JWK thumbprint. `step crypto jwk thumbprint` produces this exact identifier.

One-Liner Bash Command

```
# 1. Generate the raw RSA 2048 keypair
step crypto jwk create /tmp/pub.raw.json /tmp/priv.raw.json \
  --key RSA --size 2048 --alg RS256 --no-password --insecure

# 2. Derive the RFC 7638 SHA-256 thumbprint (base64 encoded)
KID=$(step crypto jwk thumbprint --encoding base64 /tmp/pub.raw.json)

# 3. Add 'kid' and wrap into {"keys": [...]}
jq -n --slurpfile k <(jq --arg kid "$KID" '. + {kid: $kid}' /tmp/priv.raw.json) '{"keys": $k}' > jwks.json
jq -n --slurpfile k <(jq --arg kid "$KID" '. + {kid: $kid}' /tmp/pub.raw.json) '{"keys": $k}' > jwks.pub.

# Cleanup temporary raw keys
rm -f /tmp/pub.raw.json /tmp/priv.raw.json
```

Complete Ansible Task

Here is how to automate the whole flow in an Ansible playbook using `step` and `jq`:

```
- name: Generate RS256 JWKS Key Pair
  ansible.builtin.shell: |
    set -e
    TMP_PUB=$(mktemp)
    TMP_PRIV=$(mktemp)
```

```

step crypto jwk create "$TMP_PUB" "$TMP_PRIV" \
  --key RSA \
  --size 2048 \
  --alg RS256 \
  --no-password \
  --insecure

KID=$(step crypto jwk thumbprint --encoding base64 "$TMP_PUB")

jq -n --slurpfile k <(jq --arg kid "$KID" '. + {kid: $kid}' "$TMP_PRIV") '{"keys": $k}' > /path/to/jwk
jq -n --slurpfile k <(jq --arg kid "$KID" '. + {kid: $kid}' "$TMP_PUB") '{"keys": $k}' > /path/to/jwk

chmod 0600 /path/to/jwks.json
chmod 0644 /path/to/jwks.pub.json
rm -f "$TMP_PUB" "$TMP_PRIV"
args:
creates: /path/to/jwks.json

```

Resulting Files

- **jwks.json**: Contains full RSA private parameters (key, alg, kid, n, e, d, p, q, dp, dq, qi) inside the keys list.
- **jwks.pub.json**: Contains only the public parameters (key, alg, kid, n, e) inside the keys list.

User prompt: qt.qpa.xcb: could not connect to display qt.qpa.plugin: From 6.5.0, xcb-cursor0 or libxcb-cursor0 is needed to load the Qt xcb platform plugin. qt.qpa.plugin: Could not load the Qt platform plugin "xcb" in "" even though it was found. This application failed to start because no Qt platform plugin could be initialized. Reinstalling the application may fix this problem. Available platform plugins are: eglibc, linuxfb, minimal, minimalegl, offscreen, vkkhrdisplay, vnc, wayland-brcm, wayland-egl, wayland, xcb. Aborted (core dumped)

Response: This error happens for two distinct reasons: missing underlying XCB cursor libraries required by Qt ≥ 6.5, and running in an environment without a connected graphical display (e.g., SSH, Docker, or a headless server).

Step 1: Install the Missing Dependency

Qt 6.5 and newer explicitly require `libxcb-cursor.so.0`. Install it based on your Linux distribution:

- **Debian / Ubuntu / Linux Mint:**

```

sudo apt update
sudo apt install libxcb-cursor0

```

- **Fedora / RHEL / Rocky Linux / AlmaLinux:**

```

sudo dnf install xcb-util-cursor

```

- **Arch Linux / Manjaro:**

```

sudo pacman -S xcb-util-cursor

```

Step 2: Resolve the Display Issue

The error could not connect to display indicates that the application cannot find an active X11/Wayland session. Choose the scenario that matches your environment:

Scenario A: Running Headless / in Docker / in CI/CD (No GUI Needed)

If you are generating plots (e.g., Matplotlib), processing images, or running tests without needing an interactive window, tell Qt to run offscreen:

```

export QT_QPA_PLATFORM=offscreen
python3 your_script.py # or ./your_application

```

Scenario B: Running Over SSH (GUI Forwarding)

If you are logged into a remote machine and want the GUI to show on your local computer:

1. Connect with X11 forwarding enabled:

```

ssh -X user@hostname
# or with trusted forwarding:
ssh -Y user@hostname

```

2. Verify that the DISPLAY environment variable is set on the remote machine:

```

echo $DISPLAY

```

(It should output something like localhost:10.0)

Scenario C: Running Inside Docker (Forwarding to Host GUI)

If running a container locally on Linux, grant Docker access to your X server:

```
# Allow local container connections to X11
xhost +local:root

# Run the container with the display and X11 socket mounted
docker run -it --rm \
  -e DISPLAY=$DISPLAY \
  -v /tmp/.X11-unix:/tmp/.X11-unix \
  your_image
```

Scenario D: Running on a Regular Linux Desktop

If you are in a terminal emulator on your desktop session and still see this, check if your DISPLAY variable was cleared:

```
# For standard X11:
export DISPLAY=:0

# If using Wayland, try forcing the Wayland plugin directly:
export QT_QPA_PLATFORM=wayland
```

Troubleshooting Further Missing Dependencies

If the error persists after installing libxcb-cursor0, other required X11 libraries may be missing. Run your program with Qt debugging enabled to see exactly which shared library is failing to load:

```
export QT_DEBUG_PLUGINS=1
./your_application
```

Inspect the output near the end for lines containing Cannot load library ...: (libX...: cannot open shared object file). Install any package providing the missing .so file listed there.

User prompt: [root@slurm-lab-client ~]# QT_QPA_PLATFORM=offscreen step crypto jwk create ./pup.raw.json ./priv.raw.json --kty RSA --size 2048 --alg RS256 --no-password --insecure step: Unknown options: kty, size, alg, no-password, insecure.

Response: That error means the step command on your system is not the Smallstep CLI, but rather the **KDE Step** physics simulator (a Qt desktop application from kde-edu), which is why Qt was failing to connect to an X display earlier.

Step 1: Verify Which Binary Is Being Executed

Check which step binary is mapped in your PATH:

```
which step
rpm -qf $(which step) # or dpkg -S $(which step)
```

If it shows /usr/bin/step belonging to a package like step (KDE Interactive Physical Simulator), you have the wrong tool installed.

Step 2: Install the Real Smallstep CLI

The Smallstep binary is named step, but in Linux distribution repositories it is packaged under step-cli.

On RHEL / Rocky / AlmaLinux / Fedora:

```
# Add the official Smallstep repository
cat << 'EOF' | tee /etc/yum.repos.d/smallstep.repo
[smallstep]
name=Smallstep
baseurl=https://packages.smallstep.com/rpm/
enabled=1
gpgcheck=1
repo_gpgcheck=1
gpgkey=https://packages.smallstep.com/keys/smallstep.gpg
EOF

dnf install -y step-cli
```

On Debian / Ubuntu:

```
curl -fsSL https://packages.smallstep.com/keys/smallstep.gpg | gpg --dearmor -o /etc/apt/trusted.gpg.d/sma
echo "deb [signed-by=/etc/apt/trusted.gpg.d/smallstep.gpg] https://packages.smallstep.com/apt/ stable main
apt update && apt install -y step-cli
```

Quick Static Binary (No package manager needed):

If you do not want to configure a repo on your Slurm client:

```
# Download latest Linux x86_64 binary release
curl -LO https://dl.step.sm/gh-release/cli/docs-cli-install/v0.28.6/step_linux_0.28.6_amd64.tar.gz
```

```
tar -xzf step_linux_0.28.6_amd64.tar.gz
mv step_0.28.6/bin/step /usr/local/bin/step
```

Step 3: Run the Command

Verify you are using the Smallstep version:

```
step version
# Expected: Smallstep CLI/x.y.z (...)
```

Then run your command:

```
step crypto jwk create ./pub.raw.json ./priv.raw.json \
  --key RSA \
  --size 2048 \
  --alg RS256 \
  --no-password \
  --insecure
```

User prompt: [root@slurm-lab-client ~]# step crypto jwk thumbprint --encoding base64 ./pub.raw.json Incorrect Usage: flag provided but not defined: -encoding NAME step crypto jwk thumbprint -- compute thumbprint for a JWK USAGE step crypto jwk thumbprint DESCRIPTION step crypto jwk thumbprint reads a JWK from STDINT, derives the corresponding JWK Thumbprint (RFC7638), and prints the base64-urlencoded thumbprint to STDOUT. For examples, see step help crypto jwk. flag provided but not defined: -encoding [root@slurm-lab-client ~]#

Response: In step, step crypto jwk thumbprint does not take file arguments or an --encoding flag. It expects the JWK via **standard input (STDIN)** and prints the RFC 7638 Base64URL-encoded thumbprint by default.

Command

Pass the file via redirect (<) or pipe (cat):

```
KID=$(step crypto jwk thumbprint < ./pub.raw.json)
```

Verify the derived thumbprint:

```
echo "$KID"
```

Updated Full Pipeline

```
# 1. Generate keypair
step crypto jwk create ./pub.raw.json ./priv.raw.json \
  --key RSA --size 2048 --alg RS256 --no-password --insecure

# 2. Get the RFC 7638 thumbprint from STDIN
KID=$(step crypto jwk thumbprint < ./pub.raw.json)

# 3. Add 'kid' and wrap in {"keys": [...]}
jq -n --slurpfile k <(jq --arg kid "$KID" '. + {kid: $kid}' ./priv.raw.json) '{"keys": $k}' > jwks.json
jq -n --slurpfile k <(jq --arg kid "$KID" '. + {kid: $kid}' ./pub.raw.json) '{"keys": $k}' > jwks.pub.jso

# 4. Clean up raw intermediate files
rm -f ./pub.raw.json ./priv.raw.json
```
